周报4

本周已刷题目

力扣：简单 $\times 6$ 中等 $\times 6$

牛客：中等 $\times 1$

- 堆
    1. 2208. 将数组和减半的最少操作次数 (**)
- 排序
    1. 912. 排序数组 (基数排序)(**)
- 位运算
    1. 获取最大值 *牛客题霸*牛客网 (**)
    2. 268. 丢失的数字(*)
    3. 136. 只出现一次的数字(*)
    4. 260. 只出现一次的数字 III(**)
    5. 137. 只出现一次的数字 II(**)
    6. 231. 2 的幂(*)
    7. 201. 数字范围按位与(**)
    8. 190. 颠倒二进制位(*)
    9. 461. 汉明距离(*)
- 数学
    1. 326. 3 的幂(*)
- 数组
    1. 3634. 使数组平衡的最少移除数目(**)

学习知识点

1. `<<` 是位运算符号，左移1位，`a * 2 == a << 1`；

   `>>` 是右移，`a / 2 == a << 1`；
2. 基数排序，计数排序是非比较排序，对数据的特征有要求；

   一般计数排序要求数据是非负整数且范围较窄；

   一般基数排序要求数据是非负整数；

   若不是就需要额外调整；

   排序的对象不是常规数字，改写的代价将非常大，因此非比较排序并不通用；
3. 异或运算也可以看成是无进位相加；2个相同的数异或值为0，`a ^ a == 0`；

   `sum = a ^ b`，`b = sum ^ a`；
4. `LSB (Least Significant Bit)` 算法：提取二进制数最右边的1；

   `n & (~n + 1)`；

`n & (-n)`;

5. 排序的稳定性

排序算法	时间复杂度	空间复杂度	稳定性
选择排序	$O(N^2)$	$O(1)$	不稳定
冒泡排序	$O(N^2)$	$O(1)$	稳定
插入排序	$O(N^2)$	$O(1)$	稳定
归并排序	$O(N*logN)$	$O(N)$	稳定
快速排序	$O(N*logN)$	$O(logN)$	不稳定
堆排序	$O(N*logN)$	$O(1)$	不稳定
计数排序	$O(N)$	$O(M)$	稳定
基数排序	$O(N)$	$O(M)$	稳定

6. `Brian Kernighan`算法，删除二进制最右侧的1：

`n & (n - 1)`;

笔记

- 堆
  - 2208. 将数组和减半的最少操作次数 (**)

    > 贪心策略，通过大根堆，每次把最大的元素减半，能得到最小操作次数
    >
    > 1. 从底部建堆，把`nums`中元素$\times 2^{20}$加入堆中，用`long long`存储；
    > 2. `heapify`调整堆的结构，维持大根堆；
    > 3. 对堆顶的元素减半，统计数组和减半的最小操作次数；

- 排序
  - 912. 排序数组 (基数排序)(**)

    > 对于每一位的数字进行计数排序，重复k次
    >
    > 1. 找最小值和最大值，数组中每个数减最小值，使数组元素非负
    > 2. `offset`控制当前排序的数位，统计当前位每个数字出现次数;
    >
    > ```
    > // 提取当前数位
    > cnts[(nums[i] / offset) % SYSTEM]++;
    > ```
    >
    > 3. 排序后把数写回`nums`，用于对下一位的排序
    > 4. 排序后加回最小值还原数组

- 位运算
  - 260. 只出现一次的数字 III(**)

> 1. 先求数组异或和，由于其他数出现偶数次，所以异或和为0；只有2个数出现奇数次，异或和为`a ^ b`；
> 2. `a`和`b`不同，所以`a ^ b`的结果在二进制形式下一定有为1的位；
> 3. 用`LSB`算法找数组异或和最右边的一位`1`，a和b在这一位一定不同；
> 4. 数组在这一位为0的数的异或和结果就是a和b中的一个，其他在这一位为0的数出现了偶数次，异或和为0；

- 137. 只出现一次的数字 II(**)

> 1. 统计数组在二进制每一位有多少1；
> 2. 把1的次数模m，余数不为0的位就是出现少于m次的数的二进制位；
> 3. 初始化res为0，res每位或第i位的1把res的第i位改为1，还原处少于m次的数；

```
res |= (1 << i);
```

- 201. 数字范围按位与(**)

> 1. 若left < right，(right - 1) 必然会参与按位与运算；
> 2. `right & (right - 1)`会删去`right`最右侧一位1；
> 3. 直到left和right值相等时返回right的值，就是给定区间所有数按位与的结果；

下周计划

继续学习位运算

代码

1）堆

1. 2208. 将数组和减半的最少操作次数 (**)

> 时间复杂度$O(N * logN)$;
>
> 空间复杂度$O(N)$;

```
class Solution {
public:
    enum {MAXN = 100001};
    array<long long, MAXN> heap;
    int size;
    int halveArray(vector<int>& nums) {
        int n = nums.size();
        long long sum = 0;
        size = n;
        for (int i = 0; i < n; i++) {
            // 从底建大根堆
            // 每个数* 2^20，用于控制精度
            heap[n - 1 - i] = (long long)nums[i] << 20;
            sum += heap[n - 1 - i];
            heapify(n - 1 - i);
        }
```

```
        sum /= 2;
        long long var = 0;
        int res = 0;
        while (var < sum) {
            heap[0] >>= 1; // 减半
            var += heap[0];
            heapify(0);
            res++;
        }

        return res;
    }

    // i位的数下沉
    void heapify(int i) {
        int l = 2 * i + 1;
        while (l < size) {
            // 子节点中值最大的
            int best = l + 1 < size && heap[l + 1] > heap[l] ? l + 1 : l;
            best = heap[best] > heap[i] ? best : i;
            if (best == i) {
                break;
            }
            swap(best, i);
            i = best;
            l = 2 * i + 1;
        }
    }

    void swap(int i, int j) {
        long long temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }
};
```

2）排序

1. 912. 排序数组 (基数排序)(**)

> 时间复杂度$O(k*N)$，k为最大数的位数

```
// 基数排序
class Solution {
public:
    enum {MAXN = 50001};
    enum {SYSTEM = 1000}; // 10进制,100进制.....
    array<int, MAXN> help;
    array<int, SYSTEM> cnts{};

```

```cpp
    vector<int> sortArray(vector<int>& nums) {
        if (nums.size() <= 1) {
            return nums;
        }
        int n = nums.size();
        // 找到数组中最小值
        int min = nums[0];
        for (int i = 1; i < n; i++) {
            min = nums[i] < min ? nums[i] : min;
        }
        int max = 0;
        for (int i = 0; i < n; i++) {
            // 数组中每个元素，减最小值，转化位非负数组
            nums[i] = nums[i] - min;
            max = nums[i] > max ? nums[i] : max;
        }
        // 排序，加最小值还原数字
        radixSort(nums, n, bits(max));
        for (int i = 0; i < n; i++) {
            nums[i] = nums[i] + min;
        }
        return nums;
    }

    // 返回num在SYSTEM进制下有几位
    int bits(int num) {
        int res = 0;
        while (num > 0) {
            num /= SYSTEM;
            res++;
        }
        return res;
    }

    // 基数排序，nums不能有负数,n是长度,bits是最大值在SYSTEM进制的位数
    void radixSort(vector<int>& nums, int n, int bits) {
        int offset = 1; // 先对个位排序
        while (bits > 0) {
            // 初始化cnts元素
            for (int i = 0; i < SYSTEM; i++) {
                cnts[i] = 0;
            }
            for (int i = 0; i < n; i++) {
                // 提取当前数位
                cnts[(nums[i] / offset) % SYSTEM]++;
            }
            // 前缀次数累加
            for (int i = 1; i < SYSTEM; i++) {
                cnts[i] += cnts[i - 1];
            }
            // 前缀数量分区
            for (int i = n - 1; i >= 0; i--) {
                // 提取当前数位
```

```
                help[--cnts[(nums[i] / offset) % SYSTEM]] = nums[i];
            }

            // 把排序结果写回nums，供下一数位的排序
            for (int i = 0; i < n; i++) {
                nums[i] = help[i];
            }
            bits--;
            offset *= SYSTEM;
        }
    }
};
```

3）位运算

1. 获取最大值*牛客题霸*牛客网 (**)

```
class Solution {
public:
    /**
     * 代码中的类名、方法名、参数名已经指定，请勿修改，直接返回方法规定的值即可
     *
     * 获取最大值
     * @param a int整型
     * @param b int整型
     * @return int整型
     */
    int sign(int n) {
        // 非负数返回1，负数返回0
        return ((n >> 31) & 1) ^ 1;
    }
    int getMax(int a, int b) {
        // c可能溢出
        int c = a - b;
        // 取符号
        int sa = sign(a);
        int sb = sign(b);
        int sc = sign(c);
        // 判断a和b，符号是不是不一样，如果不一样diffAB=1，如果一样diffAB=0
        int diffAB = sa ^ sb;
        // 判断a和b，符号是不是一样，如果一样sameAB=1，如果不一样sameAB=0
        int sameAB = diffAB ^ 1;

        int returnA = diffAB * sa + sameAB * sc;
        int returnB = returnA ^ 1;

        return a * returnA + b * returnB;
    }
};
```

2. 268. 丢失的数字(*)

```cpp
class Solution {
    /**
     * 0~n中共n+1个数，有n个数在数组中出现
     * 找数组中没有的数
     */
public:
    int missingNumber(vector<int>& nums) {
        int numSum = 0, iSum = 0;
        for (int i = 0; i < nums.size(); i++) {
            // 数组0~n-1元素异或和,i从0~n-1异或和
            numSum ^= nums[i];
            iSum ^= i;
        }
        // 0~n异或和异或数组中有的元素，得到缺失的
        iSum ^= nums.size();
        return iSum ^ numSum;
    }
};
```

3. 136. 只出现一次的数字(*)

```cpp
class Solution {
    /* 数组中只有一个数出现基数次，其他书都出现偶数次
     * 找出现奇数次的数
     */
public:
    int singleNumber(vector<int>& nums) {
        int dorSum = 0;
        for (int i = 0; i <nums.size(); i++) {
            // 求异或和，出现偶数次的元素异或和为0
            dorSum ^= nums[i];
        }
        return dorSum;
    }
};
```

4. 260. 只出现一次的数字 III(**)

```cpp
class Solution {
    /* 数组中只有2个数出现奇数次，其他书都出现偶数次
     * 找出现奇数次的2个数
     */
public:
    vector<int> singleNumber(vector<int>& nums) {
        // a ^ b
        int dorSum = 0;
        for (int i = 0; i < nums.size(); i++) {
            dorSum ^= nums[i];
        }

```

```
        // 提取二进制最右边的1
        // 强转为无符号整数，防溢出
        // unsigned int rightOne = (unsigned int)dorSum & ((~(unsigned int)dorSum) + 1);
        unsigned int rightOne = (unsigned int)dorSum & -(unsigned int)dorSum;

        int a = 0;
        for (int num: nums) {
            if ((num & rightOne) == 0) {
                // rightOne位上不是1的数
                a ^= num;
            }
        }
        return {a, dorSum ^ a};
    }
};
```

5. 137. 只出现一次的数字 II(**)

```
class Solution {
    /* 数组中只有1个数出现小于m次，其他书都出现m次
     * 找出现小于m次的数
     */
public:
    int singleNumber(vector<int>& nums) {
        return findNum(nums, 3);
    }

    // 找出现小于m次的数，其他数都出现了m次
    int findNum(vector<int>& nums, int m) {
        // 统计数组的二进制数每一位有几个1
        array<int,32> cnts{};
        for (int x: nums) {
            for (int i = 0; i < 32; i++) {
                // 二进制数x的第i位的1
                cnts[i] += (x >> i) & 1;
            }
        }
        // 找cnts中模m后余数不为0的
        // 即出现小于m次的数在i位是1
        int res = 0;
        for (int i = 0; i < 32; i++) {
            if (cnts[i] % m != 0) {
                // 把res的第i位改为1
                res |= (1 << i);
            }
        }
        return res;
    }
};
```

6. 231. 2 的幂(*)

```cpp
class Solution {
public:
    bool isPowerOfTwo(int n) {
        // n > 0且n在二进制下只有一个1
        return n > 0 && n == (n & -n);
    }
};
```

7. 201. 数字范围按位与(**)

```cpp
class Solution {
    /*
     * 给你两个整数 left 和 right ，表示区间 [left, right] ，
     * 返回此区间内所有数字 按位与 的结果（包含 left 、right 端点）。
     */
public:
    int rangeBitwiseAnd(int left, int right) {
        while (left < right) {
            // 删除最右侧的1
            right = right & (right - 1);
        }
        return right;
    }
};
```

8. 190. 颠倒二进制位(*)

```cpp
class Solution {
    // 颠倒给定的 32 位有符号整数的二进制位
public:
    int reverseBits(int n) {
        // 分治，先交换长度为1的部分
        n = (((unsigned int)n & 0xaaaaaaaa) >> 1) | ((n & 0x55555555) << 1); // 相邻2位交换
        n = (((unsigned int)n & 0xcccccccc) >> 2) | ((n & 0x33333333) << 2);
        n = (((unsigned int)n & 0xf0f0f0f0) >> 4) | ((n & 0x0f0f0f0f) << 4);
        n = (((unsigned int)n & 0xff00ff00) >> 8) | ((n & 0x00ff00ff) << 8);
        n = ((unsigned int)n >> 16) | (n << 16);
        return n;
    }
};
```

9. 461. 汉明距离(*)

```cpp
class Solution {
    // 返回n的二进制中有几个1
    // 两个整数之间的 汉明距离 指的是这两个数字对应二进制位不同的位置的数目
public:
    int hammingDistance(int x, int y) {
        return cntOne(x ^ y);
    }
```

```
8
9       // 统计二进制1的数目
10      int cntOne(unsigned int n) {
11          n = (n & 0x55555555) + ((n >> 1) & 0x55555555); // 统计每2位1数
12          n = (n & 0x33333333) + ((n >> 2) & 0x33333333); // 统计每4位1数
13          n = (n & 0x0f0f0f0f) + ((n >> 4) & 0x0f0f0f0f); // 统计每8位1数
14          n = (n & 0x00ff00ff) + ((n >> 8) & 0x00ff00ff); // 统计每16位1数
15          n = (n & 0x0000ffff) + ((n >> 16) & 0x0000ffff); // 统计总1数
16          return n;
17      }
18  };
```

4）数学

1. 326. 3 的幂(*)

```
1   class Solution {
2   public:
3       bool isPowerOfThree(int n) {
4           // int 范围内最大的3的幂是1162261467
5           return n > 0 && 1162261467 % n == 0;
6       }
7   };
```

5）数组

1. 3634. 使数组平衡的最少移除数目(**)

```
1   class Solution {
2   public:
3       int minRemoval(vector<int>& nums, int k) {
4           sort(nums.begin(), nums.end());
5           int n = nums.size();
6           // 滑窗
7           int res = n;
8           for (int left = 0, right = 0; left < n; left++) {
9               while (right < n && nums[right] <= (long long)k * nums[left]) {
10                  right++;
11              }
12              res = n - (right - left) < res ? n - (right - left) : res;
13          }
14          return res;
15      }
16  };
```